Análisis en Lean 4

Alex Kontorovich

Sevilla, 20 de febrero de 2026

Índice general

1 La historia del análisis real **5**

1.1 Preámbulo 5

1.2 Introducción a Lean 9

1.3 La táctica `rfl` 10

1.4 La táctica `rw` 11

1.5 La táctica `ring_nf` 12

1.6 La táctica `use` 13

1.7 La táctica `intro` 13

1.8 La táctica `specialize` 14

1.9 El desafío táctico definitivo 16

2 El cálculo de π de Newton **21**

2.1 La revolución matemática de 1666 21

2.2 La definición principal 30

3 Convergencia de sucesiones **39**

3.1 Más sobre sucesiones 39

3.2 Nivel 1: La propiedad arquimediana 39

3.2.1 La estrategia de demostración en lenguaje natural 40

3.2.2 Los desafíos de la implementación en Lean 40

3.2.3 Nuevas herramientas que necesitarás 41

3.2.4 Demostraciones en Lean 41

3.2.5 Demostración en lenguaje natural de la propiedad arquimediana . . 44

3.2.6 Revisión de errores comunes 44
3.3 Nivel 2: Nuestro primer límite real 45
3.3.1 El objetivo: Demostrar que $1/n \to 0$ 45
3.3.2 Recordatorio: La definición de convergencia de sucesiones 45
3.3.3 Estrategia de demostración en lenguaje natural 45
3.3.4 Los desafíos de la implementación en Lean 46
3.3.4.1 Desafío 1: Multiplicación cruzada de fracciones 46
3.3.4.2 Desafío 2: Aritmética lineal 46
3.3.4.3 Desafío 3: Casting explícito de tipos 46
3.3.4.4 Desafío 4: Casting en aplicaciones de tácticas 47
3.3.5 Nuevas herramientas que necesitarás 47
3.3.6 Consejos profesionales para este nivel 47
3.3.7 Demostración en Lean 47
3.3.8 Lo que acabamos de lograr 48

Capítulo 3

Convergencia de sucesiones

3.1. Más sobre sucesiones

En el Tema 2, aprendimos la definición de una sucesión $a : \mathbb{N} \to \mathbb{R}$ que converge a un límite L: para cualquier tolerancia $\varepsilon > 0$, existe un momento N, de modo que, para cualquier punto posterior a ese momento, $n \geq N$, estamos dentro de la tolerancia, $|a_n - L| < \varepsilon$.

También aprendimos cómo hacer algo completamente trivial con ella, concretamente, demostrar que la sucesión constante converge, con esa constante como su límite. Vamos a subir un peldaño, ¿de acuerdo?

3.2. Nivel 1: La propiedad arquimediana

La llamada propiedad arquimediana (que creo que se debe originalmente a Eudoxo, y aparece ya en los *Elementos* de Euclides, Libro V) es una propiedad fundamental de los números reales que captura la noción intuitiva de que no existen números reales positivos "infinitamente grandes" o "infinitamente pequeños".

Más precisamente, establece que no importa cuán pequeño sea $\varepsilon > 0$, siempre existe un número natural N de modo que $1/N$ es aún más pequeño que ε (y, por supuesto, positivo). Equivalentemente, podemos enunciarla como: para cualquier número real positivo ε, existe un número natural N tal que $1/\varepsilon < N$.

¿Por qué importa esto? La propiedad arquimediana es una de las propiedades más fundamentales que distinguen a los números reales de otros sistemas numéricos. Sin ella, podríamos tener números positivos "infinitamente grandes" o "infinitamente pequeños", lo que rompería la mayor parte del cálculo y el análisis.

Nuestro objetivo será demostrar lo siguiente:

Teorema 3.2.1 (PropArq). *Para cualquier $\varepsilon \in \mathbb{R}$ con $0 < \varepsilon$, existe $N \in \mathbb{N}$ tal que $1/\varepsilon < N$.*

Esto es matemáticamente "obvio" para la mayoría de la gente: si tienes un número positivo ε, por muy pequeño que sea, siempre puedes encontrar un número natural lo suficientemente grande para que $1/\varepsilon$ sea menor que él. Pero, ¿cómo se formaliza esto realmente en Lean?

3.2.1. La estrategia de demostración en lenguaje natural

Primero, pensemos en esto en lenguaje natural. La clave es que necesitamos proporcionar un número natural N específico que funcione.

Una opción natural sería utilizar algo relacionado con la función techo (*ceiling*). La función techo $x \mapsto \lceil x \rceil$ redondea cualquier número real al entero más cercano por arriba. Sin embargo, hay un problema sutil aquí: la función techo estándar devuelve valores en los enteros $\mathbb{Z}$, pero necesitamos valores en $\mathbb{N}$ (los números naturales). ¡¡No son lo mismo!!

Afortunadamente, Lean proporciona la "función techo de números naturales", escrita como $x \mapsto \lceil x \rceil_+$, que toma cualquier número real y devuelve un número natural. (Puedes escribir estos símbolos usando `\lceil`, `\rceil` y `\_+`). Para entradas negativas, esta función devuelve 0. Por ejemplo, $\lceil -3{,}14 \rceil_+ = 0$ y $\lceil 3{,}14 \rceil_+ = 4$.

Ahora nuestra estrategia queda clara:

- **Elección de N:** Usar $N = \lceil 1/\varepsilon \rceil_+ + 1$
- **Por qué funciona:** Tenemos la "desigualdad clave": $1/\varepsilon \leq \lceil 1/\varepsilon \rceil_+$, que se cumple por la definición de la función techo.
- **Obtener la desigualdad estricta:** Sumar 1 nos da $1/\varepsilon < \lceil 1/\varepsilon \rceil_+ + 1$

3.2.2. Los desafíos de la implementación en Lean

En Lean, los dos primeros pasos de nuestra demostración en lenguaje natural funcionan bien, pero luego nos encontramos con el problema de la **coerción de tipos** ("casting" entre diferentes tipos de números). Discutiremos esto con más detalle más adelante, pero de nuevo tiene que ver con el hecho de que $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$ y $\mathbb{R}$ son todos tipos de cosas diferentes, y necesitamos poder mover los números hacia arriba en la jerarquía de "sofisticación", siendo los naturales los objetos más simples y los reales los más complicados.

Por ejemplo, fíjate en que cuando escribamos nuestra sentencia `have` para establecer la desigualdad clave:

```
have fact : 1 / ε ≤ ⌈1 / ε⌉₊ := by WhateverTheProofIs
```

Lean la registrará como:

```
fact : 1 / ε ≤ ↑ ⌈1 / ε⌉₊
```

Observa la misteriosa flecha hacia arriba ↑. Esta representa una función de coerción de los números naturales a los reales:

$$\uparrow : \mathbb{N} \to \mathbb{R}$$

Esto se debe a que $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$ y $\mathbb{R}$ son todos tipos **diferentes** en el sistema de tipos de Lean. Aunque pensemos que los naturales están "contenidos" en los reales, formalmente son tipos distintos, y Lean necesita funciones de coerción explícitas para convertir entre ellos.

Piénsalo de esta manera: el número natural $3 : \mathbb{N}$, la fracción $3/1 : \mathbb{Q}$ y el número real $3{,}000 : \mathbb{R}$ son objetos diferentes que simplemente representan el mismo valor matemático.

La táctica `push_cast` ayuda a gestionar estas coerciones, de forma similar a `ring_nf` pero para conversiones en lugar de operaciones de anillo.

3.2.3. Nuevas herramientas que necesitarás

- $\lceil \cdot \rceil_+$: La función techo de números naturales.
- `push_cast`: Táctica que maneja las coerciones entre tipos numéricos.
- `bound`: Resuelve muchas desigualdades rutinarias una vez que los tipos están correctamente alineados.

3.2.4. Demostraciones en Lean

```
import Mathlib.Data.Real.Basic
import Mathlib.Tactic

variable {ε : ℝ}
```

```
variable (h : 0 < ε)

-- Demostración en lenguaje natural
-- ================================

-- Sea N = ⌈1/ε⌉₊ + 1. Entonces,
--    1/ε ≤ ⌈1/ε⌉₊
--        < ⌈1/ε⌉₊ + 1
--        = N
-- Por lo tanto, 1/ε < N.

-- 1ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  have fact : 1 / ε ≤ ⌈1 / ε⌉₊ := by bound
  -- fact : 1 / ε ≤ ↑⌈1 / ε⌉₊
  push_cast
  -- ⊢ 1 / ε < ↑⌈1 / ε⌉₊ + 1
  bound

-- 2ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  have fact : 1 / ε ≤ ⌈1 / ε⌉₊ := Nat.le_ceil (1 / ε)
  -- fact : 1 / ε ≤ ↑⌈1 / ε⌉₊
  push_cast
  -- ⊢ 1 / ε < ↑⌈1 / ε⌉₊ + 1
  apply lt_of_le_of_lt fact
  -- ⊢ ↑⌈1 / ε⌉₊ < ↑⌈1 / ε⌉₊ + 1
  exact lt_add_one _

-- 3ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
```

```
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  push_cast
  -- ⊢ 1 / ε < ↑⌈1 / ε⌉₊ + 1
  calc
    1 / ε ≤ ↑⌈1 / ε⌉₊     := Nat.le_ceil (1 / ε)
    _     < ↑⌈1 / ε⌉₊ + 1 := lt_add_one _

-- 4ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  push_cast
  -- ⊢ 1 / ε < ↑⌈1 / ε⌉₊ + 1
  exact lt_of_le_of_lt (Nat.le_ceil (1 / ε)) (lt_add_one _)

-- 5ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
exists_nat_gt (1 / ε)

-- 6ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  calc
    1 / ε ≤ (⌈1 / ε⌉₊ : ℝ)     := Nat.le_ceil (1 / ε)
    _     < (⌈1 / ε⌉₊ : ℝ) + 1 := lt_add_one _
    _     = ↑(⌈1 / ε⌉₊ + 1)    := by norm_num

-- Lemas usados
-- ============

variable (a b c : ℝ)
```

```
#check (Nat.le_ceil a : a ≤ ↑⌈a⌉₊)
#check (exists_nat_gt (1 / ε) : ∃ n : ℕ, 1 / ε < ↑n)
#check (lt_add_one a : a < a + 1)
#check (lt_of_le_of_lt : a ≤ b → b < c → a < c)
```

3.2.5. Demostración en lenguaje natural de la propiedad arquimediana

Teorema 3.2.2. *Para cualquier número real positivo $\varepsilon > 0$, existe un número natural N tal que $1/\varepsilon < N$.*

Demostración: Sea $\varepsilon > 0$. Necesitamos encontrar un número natural N tal que $1/\varepsilon < N$.

Usamos el valor $N = \lceil 1/\varepsilon \rceil_+ + 1$, donde $\lceil \cdot \rceil_+$ denota la función techo de números naturales.

Dado que $\varepsilon > 0$, tenemos $1/\varepsilon > 0$. Por la definición de la función techo de números naturales, sabemos que:

$$1/\varepsilon \leq \lceil 1/\varepsilon \rceil_+$$

Ahora, como $\lceil 1/\varepsilon \rceil_+$ es un número natural y $N = \lceil 1/\varepsilon \rceil_+ + 1$, tenemos:

$$\lceil 1/\varepsilon \rceil_+ < \lceil 1/\varepsilon \rceil_+ + 1 = N$$

Combinando estas desigualdades, obtenemos:

$$1/\varepsilon \leq \lceil 1/\varepsilon \rceil_+ < N$$

Por lo tanto, $1/\varepsilon < N$, lo que completa la demostración. □

Significado: La propiedad arquimediana es fundamental para el análisis porque asegura que los números reales no tienen elementos "infinitos" o "infinitesimales". Garantiza que siempre podemos encontrar números naturales lo suficientemente grandes como para dominar cualquier número real positivo dado cuando tomamos sus recíprocos. Esta propiedad es esencial para muchos procesos de límite y es equivalente a la completitud de los números reales en ciertas formulaciones.

3.2.6. Revisión de errores comunes

- ¡No uses la función techo regular $\lceil \cdot \rceil$! ¡Devuelve enteros, no naturales!
- **Cuidado con los problemas de casting**: si `bound` no funciona, prueba `push_cast` primero.

- La suma $\lceil 1/\varepsilon \rceil_+ + 1$ ocurre en $\mathbb{N}$, luego se convierte a $\mathbb{R}$; por eso necesitamos `push_cast`.

Nota histórica: Aunque a menudo se atribuye a Arquímedes (c. 287-212 a.C.), esta propiedad probablemente era conocida por Eudoxo (c. 408-355 a.C.) y aparece en los *Elementos* de Euclides. Arquímedes utilizó una versión de este principio en su método de exhaución para calcular áreas y volúmenes.

3.3. Nivel 2: Nuestro primer límite real

¡Enhorabuena! Acabas de demostrar la propiedad arquimediana. Ahora vamos a usarla para demostrar algo realmente interesante: nuestro primer límite no trivial.

3.3.1. El objetivo: Demostrar que $1/n \to 0$

Queremos demostrar que la sucesión $a(n) = 1/n$ converge a 0 cuando n tiende a infinito. Esto es intuitivamente obvio: a medida que n crece, $1/n$ se hace más pequeño y se acerca a 0. Pero, ¿cómo lo hacemos riguroso usando la definición ε-N?

Teorema 3.3.1. *La sucesión $a(n) = 1/n$ converge a 0.*

3.3.2. Recordatorio: La definición de convergencia de sucesiones

Una sucesión $a : \mathbb{N} \to \mathbb{R}$ converge a un límite L (escrito `LimSuc a L`) si:

- Para cada $\varepsilon > 0$, existe $N : \mathbb{N}$ tal que para todo $n \geq N$, $|a(n) - L| < \varepsilon$
- En notación formal: $\forall \varepsilon > 0, \exists N, \forall n \geq N, |a(n) - L| < \varepsilon$

Para nuestro caso específico con $a(n) = 1/n$ y $L = 0$, esto se convierte en: $\forall \varepsilon > 0, \exists N, \forall n \geq N, |1/n - 0| < \varepsilon$.

3.3.3. Estrategia de demostración en lenguaje natural

Así es como lo demostraremos paso a paso:

- **Paso 1:** Sea $\varepsilon > 0$ dado. (Esto corresponderá a `intro ε hε`).
- **Paso 2:** Necesitamos encontrar un N tal que para todo $n \geq N$, tengamos $1/n < \varepsilon$.

- **Idea clave:** Necesitamos $1/n < \varepsilon$, que es equivalente a $1/\varepsilon < n$ (ya que ambos lados son positivos). Así que necesitamos que n sea mayor que $1/\varepsilon$.
- **Paso 3:** ¡Para eso desarrollamos la propiedad arquimediana! Nos dice que existe algún número natural N tal que $1/\varepsilon < N$. En lugar de volver a demostrarlo desde cero, podemos simplemente citar este hecho; luego elegiremos tal N y lo usaremos.
- **Paso 4:** Ahora sea $n \geq N$ dado. Tenemos:
  - $1/\varepsilon < N$ (por nuestra elección de N)
  - $N \leq n$ (por hipótesis)
  - Por lo tanto: $1/\varepsilon < N \leq n$, así que $1/\varepsilon < n$
  - Tomando recíprocos (y volteando la desigualdad): $1/n < \varepsilon$
- **Paso 5:** Como $|1/n - 0| = |1/n| = 1/n < \varepsilon$, ¡hemos terminado!

3.3.4. Los desafíos de la implementación en Lean

3.3.4.1. Desafío 1: Multiplicación cruzada de fracciones

Nuestro paso clave es mostrar que $1/n < \varepsilon$. En matemáticas de papel, simplemente multiplicaríamos de forma cruzada para obtener $1 < n * \varepsilon$. Pero Lean es muy cuidadoso con la división por cero. **Solución:** ¡La táctica `field_simp` maneja esto automáticamente! Eliminará denominadores multiplicando de forma cruzada, pero solo después de verificar que todos los denominadores son positivos (o al menos no nulos).

3.3.4.2. Desafío 2: Aritmética lineal

Una vez eliminadas las fracciones, necesitamos combinar varias desigualdades. La táctica `bound` no siempre maneja bien estas combinaciones lineales con multiplicación por variables. **Solución:** La táctica `linarith` está específicamente diseñada para la aritmética lineal. Puede tomar una lista de hipótesis y resolver metas que se sigan de combinaciones lineales de esas hipótesis.

3.3.4.3. Desafío 3: Casting explícito de tipos

¿Recuerdas las flechas hacia arriba ↑? ¡Han vuelto! Cuando escribimos $1/n$, Lean lo ve como $1/\uparrow n$ donde n comienza como un número natural pero necesita ser convertido a real. **Solución:** Podemos especificar el casting explícitamente con esta sintaxis: `(n : ℝ)`. Esto le dice a Lean exactamente a qué tipo queremos convertir n.

3.3.4.4. Desafío 4: Casting en aplicaciones de tácticas

A veces quieres aplicar una táctica pero los tipos no coinciden exactamente. **Solución:** La táctica `exact_mod_cast` es como `apply`, pero maneja automáticamente las coerciones de tipo por ti.

3.3.5. Nuevas herramientas que necesitarás

- `field_simp`: Elimina denominadores.
- `linarith`: Potente táctica para desigualdades lineales.
- `exact_mod_cast`: Maneja coerciones automáticamente cuando la meta es exactamente la hipótesis tras el casting.

3.3.6. Consejos profesionales para este nivel

1. Usa `change` para convertir `LimSuc a 0` a su definición.
2. Aplica la propiedad arquimediana para obtener la existencia de N.
3. Usa `choose` para extraer el N de la sentencia existencial.
4. Establece la positividad primero antes de usar `field_simp`.
5. Trabaja paso a paso.

3.3.7. Demostración en Lean

```
Statement OneOverNLimZero ( a : $\mathbb{N}$ $\to$ $\mathbb{R}$) ( ha : $\forall$ n , a n
↪  = 1 / n ) :
  LimSuc a 0 := by
  change $\forall$ $\varepsilon$ > 0 , $\exists$ N , $\forall$ n $\ge$ N , | a n - 0| <
↪  $\varepsilon$
  intro $\varepsilon$ h$\varepsilon$
  have f1 : $\exists$ ( N : $\mathbb{N}$) , 1 / $\varepsilon$ < N := by apply PropArq
↪  h$\varepsilon$
  choose N eps_inv_lt_N using f1
  use N
  intro n n_ge_N
  ring_nf
  specialize ha n
```

```
rewrite [ ha ]
have f2 : |1 / ( n : $\mathbb{R}$) | = 1 / n := by bound
rewrite [ f2 ]
have f3 : 0 < 1 / $\varepsilon$ := by bound
have Npos : (0 : $\mathbb{R}$) < N := by linarith [ f3 , eps_inv_lt_N ]
have N_le_n : ( N : $\mathbb{R}$) $\le$ n := by exact_mod_cast n_ge_N
have npos : (0 : $\mathbb{R}$) < n := by linarith [ Npos , N_le_n ]
field_simp
field_simp at eps_inv_lt_N
have f4 : N * $\varepsilon$ $\le$ n * $\varepsilon$ := by bound
linarith [ eps_inv_lt_N , f4 ]
```

3.3.8. Lo que acabamos de lograr

Esta demostración demuestra varios conceptos clave:

1. El poder de la propiedad arquimediana.
2. La definición ε-N en acción.
3. Manipulación rigurosa de desigualdades.
4. El puente entre la intuición y la formalidad.